

Laporan Praktikum

Kecerdasan Komputasional (Probability and Uncertainty)

Identitas Praktikan

Nama	NRP	Kelas	Jurusan
Wayan Raditya Putra	5054241029	N	Rekayasa Kecerdasan Artifisial

Dosen Pengampu

Nama Dosen
Imam Mustafa Kamal, S.ST, Ph.D

Judul Praktikum

Representasi Ketidakpastian Menggunakan Probabilitas dan Distribusi

Tujuan

Mempelajari bagaimana probabilitas digunakan untuk merepresentasikan ketidakpastian dalam sistem kecerdasan komputasional, serta memahami penerapan konsep dasar seperti variabel acak, distribusi peluang, dan inferensi probabilistik.

Inisialisasi dan Import Library

Pada bagian ini dilakukan proses *import* terhadap beberapa modul yang digunakan dalam praktikum.

Baris kode berikut:

```
In [1]: from probability import *
        from utils import print_table
        from notebook import psource, pseudocode, heatmap
```

```
d:\AI_Journey\Computational_Intelligence\CI\Lib\site-packages\qpsolvers\solvers\_
_init__.py:880: UserWarning: no QP solver found on your system, you can install s
olvers from PyPI by ``pip install qpsolvers[open_source_solvers]``
  warnings.warn(
```

PROBABILITY DISTRIBUTION

```
In [2]: psource(ProbDist)
```

class ProbDist:

"""A discrete probability distribution. You name the random variable in the constructor, then assign and query probability of values.

>>> P = ProbDist('Flip'); P['H'], P['T'] = 0.25, 0.75; P['H']

0.25

>>> P = ProbDist('X', {'lo': 125, 'med': 375, 'hi': 500})

>>> P['lo'], P['med'], P['hi']

(0.125, 0.375, 0.5)

"""

def __init__(self, var_name='?', freq=None):

"""If freq is given, it is a dictionary of values - frequency pairs, then ProbDist is normalized."""

self.prob = {}

self.var_name = var_name

self.values = []

if freq:

for (v, p) in freq.items():

self[v] = p

self.normalize()

def __getitem__(self, val):

"""Given a value, return P(value)."""

try:

return self.prob[val]

except KeyError:

return 0

def __setitem__(self, val, p):

"""Set P(val) = p."""

if val not in self.values:

self.values.append(val)

self.prob[val] = p

def normalize(self):

"""Make sure the probabilities of all values sum to 1.

Returns the normalized distribution.

Raises a ZeroDivisionError if the sum of the values is 0."""

total = sum(self.prob.values())

if not np.isclose(total, 1.0):

for val in self.prob:

self.prob[val] /= total

return self

```

def show_approx(self, numfmt='{:.3g}'):
    """Show the probabilities rounded and sorted by key, for the
    sake of portable doctests."""
    return ', '.join(['{}: ' + numfmt.format(v, p) for (v, p) in sorted(self.prob.items())])

def __repr__(self):
    return "P({})".format(self.var_name)

```

Kelas ProbDist

Bagian ini mendefinisikan kelas ProbDist yang berfungsi untuk merepresentasikan distribusi probabilitas diskret (discrete probability distribution) dari sebuah variabel acak.

Distribusi probabilitas diskret menggambarkan kumpulan nilai-nilai yang mungkin dari suatu variabel acak beserta peluang kemunculannya, di mana total seluruh peluang bernilai 1.

Penjelasan fungsi dan atribut utama:

1. ******init(self, var_name='?', freq=None)** Konstruktor kelas yang menginisialisasi objek ProbDist. Parameter var_name adalah nama variabel acak (misalnya Flip, X, dan sebagainya). Parameter freq bersifat opsional dan berisi pasangan nilai-frekuensi dalam bentuk dictionary. Jika diberikan, distribusi akan otomatis dinormalisasi agar total probabilitasnya menjadi 1.
2. ******getitem(self, val)** Mengembalikan nilai probabilitas untuk suatu value tertentu, misalnya P['H']. Jika nilai belum pernah ditentukan, akan dikembalikan 0.
3. ******setitem(self, val, p)** Menetapkan probabilitas p untuk sebuah nilai val. Jika val belum ada dalam daftar nilai, otomatis ditambahkan ke atribut values.
4. **normalize(self)** Melakukan normalisasi, yaitu memastikan total semua probabilitas berjumlah 1. Hal ini penting karena dalam teori probabilitas, $\sum P(x) = 1$ merupakan syarat distribusi valid. Jika total awal bukan 1, semua nilai akan dibagi dengan totalnya agar memenuhi syarat tersebut.
5. **show_approx(self, numfmt='{:.3g}')** Menampilkan isi distribusi dalam format teks yang rapi dan dibulatkan (rounded), berguna untuk dokumentasi atau pengujian.
6. ******repr(self)** Menampilkan representasi objek dalam format P(var_name), sehingga memudahkan pembacaan saat objek dipanggil di interpreter.

Kesimpulannya, kelas ProbDist berperan sebagai pondasi dasar dalam representasi probabilistik di notebook ini. Kelas ini digunakan untuk menyimpan, mengakses, dan menormalkan nilai peluang secara terstruktur serta mudah dioperasikan pada eksperimen berikutnya.

```

In [3]: p = ProbDist('Flip')
        p['H'], p['T'] = 0.25, 0.75
        p['T']

```

Out[3]: 0.75

Potongan kode di atas digunakan untuk membuat dan menguji objek dari kelas ProbDist. Objek bernama p dideklarasikan dengan variabel acak "Flip" yang merepresentasikan percobaan lemparan koin. Selanjutnya ditetapkan nilai probabilitas untuk dua kemungkinan hasil, yaitu sisi "H" (Head) sebesar 0.25 dan sisi "T" (Tail) sebesar 0.75. Baris terakhir p['T'] berfungsi untuk menampilkan nilai probabilitas dari sisi "T", yaitu 0.75. Dengan demikian, kode ini menunjukkan cara mendefinisikan distribusi probabilitas sederhana dan mengakses nilai peluang dari suatu variabel acak.

```
In [4]: p = ProbDist(freq={'low': 125, 'medium': 375, 'high': 500})
p.var_name
```

Out[4]: '?'

Potongan kode ini membuat objek distribusi probabilitas bernama p menggunakan parameter freq yang berisi pasangan nilai-frekuensi, yaitu 'low' sebesar 125, 'medium' sebesar 375, dan 'high' sebesar 500. Pada saat objek dibuat, konstruktor kelas ProbDist secara otomatis akan menormalkan frekuensi tersebut sehingga total probabilitasnya menjadi 1. Setelah itu, baris p.var_name digunakan untuk menampilkan nama variabel acak yang dimiliki objek tersebut. Karena pada inisialisasi tidak diberikan nama variabel, maka nilai default yang ditampilkan adalah tanda tanya (?) yang berasal dari parameter var_name di konstruktor.

```
In [5]: (p['low'], p['medium'], p['high'])
```

Out[5]: (0.125, 0.375, 0.5)

Potongan kode ini digunakan untuk menampilkan nilai probabilitas dari masing-masing kategori yaitu 'low', 'medium', dan 'high' pada objek distribusi probabilitas p yang telah dibuat sebelumnya. Karena frekuensi awal yang dimasukkan adalah 125, 375, dan 500, maka setelah dilakukan normalisasi totalnya menjadi 1000, dan masing-masing nilai probabilitas adalah $P(\text{low}) = 125/1000 = 0.125$, $P(\text{medium}) = 375/1000 = 0.375$, dan $P(\text{high}) = 500/1000 = 0.5$. Dengan demikian, ketika kode (p['low'], p['medium'], p['high']) dijalankan, hasil yang ditampilkan adalah (0.125, 0.375, 0.5).

```
In [6]: p.values
```

Out[6]: ['low', 'medium', 'high']

Potongan kode ini digunakan untuk menampilkan daftar nilai atau kategori yang terdapat dalam objek distribusi probabilitas p. Atribut values menyimpan semua kemungkinan nilai dari variabel acak yang telah dimasukkan, yaitu 'low', 'medium', dan 'high'. Dengan demikian, ketika p.values dipanggil, hasil yang ditampilkan adalah daftar ['low', 'medium', 'high'] yang merepresentasikan seluruh nilai yang memiliki probabilitas dalam distribusi tersebut.

```
In [7]: p = ProbDist('Y')
p['Cat'] = 50
p['Dog'] = 114
```

```
p['Mice'] = 64
(p['Cat'], p['Dog'], p['Mice'])
```

Out[7]: (50, 114, 64)

```
In [8]: p.normalize()
(p['Cat'], p['Dog'], p['Mice'])
```

Out[8]: (0.21929824561403508, 0.5, 0.2807017543859649)

Potongan kode ini membuat objek distribusi probabilitas bernama `p` dengan variabel acak "Y". Nilai-nilai yang mungkin untuk variabel tersebut adalah "Cat", "Dog", dan "Mice", masing-masing diberi frekuensi awal sebesar 50, 114, dan 64. Pada tahap pertama, ketika `(p['Cat'], p['Dog'], p['Mice'])` dipanggil, hasil yang ditampilkan masih berupa frekuensi mentah, bukan probabilitas, yaitu (50, 114, 64). Setelah itu, fungsi `normalize()` dijalankan untuk menormalkan total frekuensi menjadi 1. Proses ini dilakukan dengan membagi setiap frekuensi dengan total keseluruhan ($50 + 114 + 64 = 228$). Dengan demikian, nilai probabilitas hasil normalisasi adalah $P(\text{Cat}) = 50/228 \approx 0.219$, $P(\text{Dog}) = 114/228 \approx 0.5$, dan $P(\text{Mice}) = 64/228 \approx 0.281$. Ketika `(p['Cat'], p['Dog'], p['Mice'])` dipanggil kembali setelah normalisasi, hasil yang ditampilkan mencerminkan probabilitas tersebut.

```
In [9]: p.show_approx()
```

Out[9]: 'Cat: 0.219, Dog: 0.5, Mice: 0.281'

Potongan kode ini digunakan untuk menampilkan isi dari distribusi probabilitas `p` dalam format teks yang sudah dibulatkan dan disusun berdasarkan urutan nama kategorinya. Fungsi `show_approx()` mempermudah pembacaan hasil karena menampilkan setiap nilai beserta probabilitasnya dalam satu baris yang rapi. Berdasarkan hasil normalisasi sebelumnya, output yang ditampilkan adalah "Cat: 0.219, Dog: 0.5, Mice: 0.281", yang menunjukkan bahwa peluang terbesar terdapat pada kategori "Dog" dengan probabilitas 0.5.

Joint Probability Distribution

```
In [10]: event = {'A': 10, 'B': 9, 'C': 8}
variables = ['C', 'A']
event_values(event, variables)
```

Out[10]: (8, 10)

Potongan kode ini membuat sebuah dictionary bernama `event` yang merepresentasikan kumpulan pasangan variabel dan nilainya, yaitu $A = 10$, $B = 9$, dan $C = 8$. Kemudian dibuat sebuah list bernama `variables` yang berisi dua variabel yang ingin diambil nilainya, yaitu 'C' dan 'A'. Fungsi `event_values(event, variables)` digunakan untuk mengekstrak nilai dari variabel-variabel tersebut dari dictionary `event` sesuai urutan yang diberikan. Hasil yang dikembalikan adalah tuple (8, 10), karena nilai variabel 'C' adalah 8 dan nilai variabel 'A' adalah 10.

```
In [11]: psource(JointProbDist)
```

```
class JointProbDist(ProbDist):
    """A discrete probability distribute over a set of variables.
    >>> P = JointProbDist(['X', 'Y']); P[1, 1] = 0.25
    >>> P[1, 1]
    0.25
    >>> P[dict(X=0, Y=1)] = 0.5
    >>> P[dict(X=0, Y=1)]
    0.5"""

    def __init__(self, variables):
        self.prob = {}
        self.variables = variables
        self.vals = defaultdict(list)

    def __getitem__(self, values):
        """Given a tuple or dict of values, return P(values)."""
        values = event_values(values, self.variables)
        return ProbDist.__getitem__(self, values)

    def __setitem__(self, values, p):
        """Set P(values) = p. Values can be a tuple or a dict; it must
        have a value for each of the variables in the joint. Also keep track
        of the values we have seen so far for each variable."""
        values = event_values(values, self.variables)
        self.prob[values] = p
        for var, val in zip(self.variables, values):
            if val not in self.vals[var]:
                self.vals[var].append(val)

    def values(self, var):
        """Return the set of possible values for a variable."""
        return self.vals[var]

    def __repr__(self):
        return "P({})".format(self.variables)
```

Kelas JointProbDist merupakan turunan dari kelas ProbDist yang digunakan untuk merepresentasikan distribusi probabilitas gabungan (joint probability distribution) dari beberapa variabel acak sekaligus. Distribusi ini tidak hanya memuat peluang dari satu variabel, tetapi kombinasi nilai dari dua atau lebih variabel yang saling berhubungan.

Konstruktor **init**(self, variables) menginisialisasi objek dengan tiga atribut utama, yaitu prob untuk menyimpan pasangan nilai dan probabilitas, variables untuk menampung

daftar variabel yang terlibat dalam distribusi, serta vals yang berfungsi menyimpan daftar nilai yang mungkin untuk setiap variabel menggunakan struktur defaultdict(list).

Metode **getitem**(self, values) digunakan untuk mengembalikan nilai probabilitas suatu kombinasi variabel. Parameter values dapat berupa tuple atau dictionary. Fungsi event_values akan mengekstrak nilai-nilai sesuai urutan variabel dalam distribusi, lalu memanggil fungsi milik kelas induk ProbDist untuk mendapatkan nilai peluangnya.

Metode **setitem**(self, values, p) digunakan untuk menetapkan nilai probabilitas p pada kombinasi nilai tertentu. Parameter values bisa berupa tuple atau dictionary yang berisi nilai untuk masing-masing variabel. Selain menyimpan pasangan nilai dan probabilitas ke dalam atribut prob, fungsi ini juga memperbarui daftar nilai yang sudah muncul untuk setiap variabel pada atribut vals.

Metode values(self, var) berfungsi untuk mengembalikan daftar nilai yang mungkin untuk satu variabel tertentu dalam distribusi gabungan.

Sementara itu, metode **repr**(self) memberikan representasi string dari objek dalam format P(variables) agar mudah dibaca di interpreter.

Sebagai contoh, jika dibuat distribusi gabungan antara variabel X dan Y menggunakan kode `P = JointProbDist(['X', 'Y']); P[1, 1] = 0.25`, maka objek P menyimpan peluang gabungan $P(X=1, Y=1)$ sebesar 0.25. Dengan demikian, kelas ini memungkinkan representasi probabilitas lebih kompleks yang melibatkan hubungan antarvariabel dalam suatu sistem acak.

```
In [12]: variables = ['X', 'Y']
         j = JointProbDist(variables)
         j
```

```
Out[12]: P(['X', 'Y'])
```

Potongan kode ini digunakan untuk membuat objek distribusi probabilitas gabungan bernama j dengan dua variabel acak yaitu "X" dan "Y". Baris pertama mendefinisikan daftar variables yang berisi nama kedua variabel tersebut. Kemudian, baris kedua membuat objek JointProbDist menggunakan daftar variabel tersebut sebagai argumen. Ketika objek j dipanggil, Python akan menjalankan metode **repr** yang telah didefinisikan dalam kelas JointProbDist, sehingga hasil yang ditampilkan adalah representasi string "P(['X', 'Y'])". Hal ini menunjukkan bahwa objek j merupakan distribusi probabilitas gabungan untuk variabel X dan Y, namun belum memiliki pasangan nilai dan probabilitas di dalamnya.

```
In [13]: j[1,1] = 0.2
         j[dict(X=0, Y=1)] = 0.5

         (j[1,1], j[0,1])
```

```
Out[13]: (0.2, 0.5)
```

Potongan kode ini menunjukkan cara penggunaan metode khusus (magic methods) dalam kelas `JointProbDist` untuk menetapkan dan mengakses nilai probabilitas dari kombinasi variabel. Objek `j` yang telah dibuat sebelumnya digunakan untuk menyimpan distribusi gabungan antara variabel `X` dan `Y`. Pada baris `j[1,1] = 0.2`, probabilitas gabungan $P(X=1, Y=1)$ ditetapkan sebesar 0.2 menggunakan format tuple. Sedangkan baris `j[dict(X=0, Y=1)] = 0.5` menggunakan format dictionary untuk menetapkan nilai probabilitas $P(X=0, Y=1)$ sebesar 0.5. Pemanggilan fungsi `event_values` di dalam metode **`getitem`** dan **`setitem`** berfungsi untuk mengekstrak nilai-nilai sesuai urutan variabel yang telah ditentukan, sehingga kedua format penulisan tersebut dapat diproses dengan cara yang sama. Baris terakhir (`j[1,1]`, `j[0,1]`) menampilkan nilai probabilitas dari dua kombinasi variabel tersebut, yaitu 0.2 untuk $(X=1, Y=1)$ dan 0.5 untuk $(X=0, Y=1)$. Dengan demikian, kode ini memperlihatkan fleksibilitas kelas `JointProbDist` dalam menerima berbagai format input untuk representasi probabilitas gabungan.

```
In [14]: j.values('X')
```

```
Out[14]: [1, 0]
```

Potongan kode ini digunakan untuk menampilkan semua nilai yang mungkin dimiliki oleh variabel tertentu dalam distribusi probabilitas gabungan. Fungsi `j.values('X')` memanggil metode `values` pada objek `JointProbDist` untuk variabel "`X`". Berdasarkan data yang telah dimasukkan sebelumnya, variabel `X` memiliki dua nilai yang pernah muncul, yaitu 0 dan 1, karena pada baris sebelumnya telah ditetapkan probabilitas untuk pasangan $(X=1, Y=1)$ dan $(X=0, Y=1)$. Oleh karena itu, hasil dari perintah ini adalah daftar `[0, 1]`. Dengan kata lain, fungsi `values` digunakan untuk mengetahui domain atau ruang kemungkinan dari suatu variabel acak dalam distribusi gabungan.

Inferensi Menggunakan Full Joint Distribution

Bagian ini membahas penerapan inferensi menggunakan **distribusi probabilitas gabungan penuh (Full Joint Distribution)** untuk menghitung **distribusi posterior** berdasarkan bukti (*evidence*) yang diberikan. Bukti direpresentasikan dalam bentuk *dictionary* Python, di mana kunci menunjukkan nama variabel dan nilainya menunjukkan kondisi yang diamati.

Pendekatan ini mengacu pada teori yang dijelaskan dalam **Section 13.3** buku *Artificial Intelligence: A Modern Approach* (AIMA). Inferensi dilakukan dengan cara menghitung probabilitas bersyarat menggunakan enumerasi dari distribusi gabungan penuh, sebagaimana dijelaskan dalam **Persamaan 13.9** berikut:

$$P(X \mid \mathbf{e}) = \alpha P(X, \mathbf{e}) = \alpha \sum_y P(X, \mathbf{e}, y)$$

Keterangan:

- **α (alpha)** adalah faktor normalisasi agar total probabilitas hasil bernilai 1.
- **X** adalah variabel yang ingin dicari probabilitasnya (*query variable*).
- **$\mathbf{e}$** adalah bukti (*evidence*), yaitu variabel yang nilainya diketahui.

- **y** adalah variabel lain yang tidak termasuk dalam evidence maupun query variable.

Dengan kata lain, inferensi dilakukan dengan menjumlahkan semua kombinasi nilai dari variabel **y** untuk memperoleh probabilitas bersyarat **P(X | e)**.

Selanjutnya, dibuat distribusi probabilitas gabungan penuh bernama `full_joint` yang melibatkan tiga variabel: `Cavity`, `Toothache`, dan `Catch`. Setiap baris kode menetapkan probabilitas untuk kombinasi nilai dari ketiga variabel tersebut, misalnya:

$$P(Cavity = True, Toothache = True, Catch = True) = 0.108$$

dan seterusnya.

Data ini diambil dari **Gambar 13.3** dalam buku yang sama, dan merepresentasikan keseluruhan ruang probabilitas dari semua kombinasi nilai ketiga variabel tersebut.

Distribusi ini akan digunakan untuk proses inferensi probabilistik pada bagian berikutnya.

```
In [15]: full_joint = JointProbDist(['Cavity', 'Toothache', 'Catch'])
full_joint[dict(Cavity=True, Toothache=True, Catch=True)] = 0.108
full_joint[dict(Cavity=True, Toothache=True, Catch=False)] = 0.012
full_joint[dict(Cavity=True, Toothache=False, Catch=True)] = 0.016
full_joint[dict(Cavity=True, Toothache=False, Catch=False)] = 0.064
full_joint[dict(Cavity=False, Toothache=True, Catch=True)] = 0.072
full_joint[dict(Cavity=False, Toothache=False, Catch=True)] = 0.144
full_joint[dict(Cavity=False, Toothache=True, Catch=False)] = 0.008
full_joint[dict(Cavity=False, Toothache=False, Catch=False)] = 0.576
```

```
In [16]: psource(enumerate_joint)
```

```
def enumerate_joint(variables, e, P):
    """Return the sum of those entries in P consistent with e,
    provided variables is P's remaining variables (the ones not in e)."""
    if not variables:
        return P[e]
    Y, rest = variables[0], variables[1:]
    return sum([enumerate_joint(rest, extend(e, Y, y), P) for y in P.values(Y)])
```

Fungsi `enumerate_joint` digunakan untuk menghitung total probabilitas dari semua entri dalam distribusi probabilitas gabungan penuh (Full Joint Distribution) yang konsisten dengan bukti atau evidence **e**. Fungsi ini bekerja secara rekursif untuk menjumlahkan seluruh kombinasi nilai variabel yang belum diketahui. Parameter `variables` berisi daftar variabel yang tersisa, yaitu variabel yang belum termasuk dalam evidence, sedangkan **P** merupakan objek distribusi probabilitas gabungan yang digunakan untuk mengambil nilai probabilitas dari kombinasi nilai tertentu. Jika daftar `variables` sudah kosong, maka fungsi langsung mengembalikan nilai probabilitas **P[e]**, yaitu probabilitas dari kombinasi nilai yang sesuai dengan evidence yang ada. Namun, jika masih ada variabel tersisa, fungsi akan mengambil variabel pertama dari daftar (diberi nama **Y**) dan memanggil dirinya sendiri secara rekursif untuk setiap kemungkinan nilai dari variabel tersebut. Nilai-nilai ini diambil dari `P.values(Y)`, yaitu daftar nilai yang mungkin dari variabel **Y**. Setiap iterasi akan memperluas evidence dengan menambahkan pasangan variabel dan nilainya

menggunakan fungsi `extend(e, Y, y)`, kemudian hasil dari setiap pemanggilan rekursif dijumlahkan menggunakan fungsi `sum`. Secara keseluruhan, fungsi `enumerate_joint` menerapkan prinsip enumerasi terhadap semua kombinasi nilai variabel yang konsisten dengan `evidence`, sebagaimana dijelaskan dalam teori inferensi probabilistik pada distribusi gabungan penuh.

```
In [17]: evidence = dict(Toothache=True)
variables = ['Cavity', 'Catch'] # variables not part of evidence
ans1 = enumerate_joint(variables, evidence, full_joint)
ans1
```

Out[17]: 0.19999999999999998

Potongan kode ini digunakan untuk menghitung total probabilitas dari semua kombinasi variabel yang konsisten dengan bukti (`evidence`) yang diberikan menggunakan fungsi `enumerate_joint`. Bukti yang digunakan adalah bahwa `Toothache` bernilai `True`, yang disimpan dalam dictionary `evidence = dict(Toothache=True)`. Variabel lain yang belum termasuk dalam `evidence`, yaitu `Cavity` dan `Catch`, disimpan dalam list `variables`. Pemanggilan fungsi `enumerate_joint(variables, evidence, full_joint)` kemudian akan menghitung jumlah dari semua probabilitas dalam distribusi gabungan penuh `full_joint` yang sesuai dengan kondisi `Toothache=True`. Proses ini dilakukan dengan menjumlahkan seluruh nilai $P(\text{Cavity}, \text{Catch}, \text{Toothache}=\text{True})$ untuk semua kombinasi nilai `Cavity` dan `Catch`. Hasil akhirnya disimpan dalam variabel `ans1` dan mewakili nilai total probabilitas terjadinya `Toothache=True` tanpa memperhatikan kondisi variabel lain.

```
In [18]: evidence = dict(Cavity=True, Toothache=True)
variables = ['Catch'] # variables not part of evidence
ans2 = enumerate_joint(variables, evidence, full_joint)
ans2
```

Out[18]: 0.12

Potongan kode ini digunakan untuk menghitung probabilitas gabungan antara dua variabel, yaitu $P(\text{Cavity}=\text{True} \text{ dan } \text{Toothache}=\text{True})$, dengan menggunakan fungsi `enumerate_joint` yang telah dibuat sebelumnya. Bukti atau `evidence` yang diberikan berisi dua kondisi, yaitu `Cavity=True` dan `Toothache=True`, yang disimpan dalam dictionary `evidence = dict(Cavity=True, Toothache=True)`. Variabel yang belum termasuk dalam `evidence` adalah `Catch`, sehingga dimasukkan ke dalam list `variables`. Ketika fungsi `enumerate_joint(variables, evidence, full_joint)` dijalankan, fungsi akan menghitung jumlah semua probabilitas dari distribusi gabungan penuh `full_joint` yang sesuai dengan kondisi tersebut, yaitu dengan menjumlahkan semua nilai $P(\text{Cavity}=\text{True}, \text{Toothache}=\text{True}, \text{Catch}=y)$ untuk seluruh kemungkinan nilai y dari variabel `Catch` (`True` dan `False`). Hasil akhirnya disimpan dalam variabel `ans2`, yang menunjukkan total probabilitas terjadinya kondisi `Cavity=True` dan `Toothache=True` secara bersamaan, tanpa memperhatikan nilai variabel `Catch`.

```
In [19]: ans2/ans1
```

Out[19]: 0.6

Potongan kode ini digunakan untuk menghitung probabilitas bersyarat **P(Cavity=True | Toothache=True)**, yaitu peluang seseorang memiliki kavitas gigi (Cavity=True) dengan asumsi bahwa orang tersebut mengalami sakit gigi (Toothache=True). Berdasarkan definisi dasar probabilitas bersyarat, rumusnya dapat dituliskan sebagai berikut:

$$P(Cavity = True \mid Toothache = True) = \frac{P(Cavity = True \text{ and } Toothache = True)}{P(Toothache = True)}$$

Pada bagian sebelumnya, nilai **P(Cavity=True and Toothache=True)** telah dihitung dan disimpan dalam variabel `ans2`, sedangkan nilai **P(Toothache=True)** disimpan dalam `ans1`. Dengan demikian, operasi `ans2 / ans1` digunakan untuk menghitung hasil pembagian antara keduanya. Hasil akhirnya merepresentasikan probabilitas seseorang memiliki kavitas gigi dengan kondisi diketahui bahwa ia mengalami sakit gigi, sesuai dengan teori inferensi probabilistik pada distribusi gabungan penuh.



```
In [20]: psource(enumerate_joint_ask)
```

```
def enumerate_joint_ask(X, e, P):
    """
    [Section 13.3]
    Return a probability distribution over the values of the variable X,
    given the {var:val} observations e, in the JointProbDist P.
    >>> P = JointProbDist(['X', 'Y'])
    >>> P[0,0] = 0.25; P[0,1] = 0.5; P[1,1] = P[2,1] = 0.125
    >>> enumerate_joint_ask('X', dict(Y=1), P).show_approx()
    '0: 0.667, 1: 0.167, 2: 0.167'
    """
    assert X not in e, "Query variable must be distinct from evidence"
    Q = ProbDist(X) # probability distribution for X, initially empty
    Y = [v for v in P.variables if v != X and v not in e] # hidden variables.
    for xi in P.values(X):
        Q[xi] = enumerate_joint(Y, extend(e, X, xi), P)
    return Q.normalize()
```

Fungsi **enumerate_joint_ask** digunakan untuk menghitung distribusi probabilitas penuh atas semua kemungkinan nilai dari variabel kueri **X**, dengan kondisi bukti atau *evidence* **e** pada suatu distribusi probabilitas gabungan (**JointProbDist**) **P**. Langkah pertama pada fungsi ini adalah memastikan bahwa variabel **X** tidak termasuk dalam *evidence*, karena variabel yang sedang ditanyakan tidak boleh sekaligus dijadikan bukti. Setelah itu, fungsi membuat objek **Q** bertipe **ProbDist(X)** yang berfungsi sebagai wadah kosong untuk menyimpan distribusi probabilitas dari variabel **X**. Daftar **Y** kemudian dibuat untuk menampung *hidden variables*, yaitu variabel-variabel yang ada di dalam **P** tetapi tidak termasuk dalam **X** maupun **evidence**. Untuk setiap kemungkinan nilai **xi** dari variabel **X** (yang diambil menggunakan **P.values(X)**), fungsi akan memanggil **enumerate_joint** dengan memperluas *evidence* menggunakan pasangan baru (**X = xi**) melalui fungsi

extend(e, X, xi). Hasil dari setiap pemanggilan **enumerate_joint** akan disimpan ke dalam **Q[xi]**, yang berarti probabilitas dari nilai **xi** diberikan bukti **e**. Setelah semua nilai **xi** selesai dihitung, fungsi melakukan normalisasi dengan **Q.normalize()** agar total seluruh probabilitas dalam distribusi **Q** bernilai satu. Dengan demikian, hasil akhirnya adalah distribusi bersyarat **P(X | e)** yang menggambarkan seberapa besar kemungkinan setiap nilai **X** terjadi berdasarkan bukti yang diberikan. Contoh penggunaan pada *doctest* menunjukkan bahwa ketika **e = {Y = 1}**, fungsi menghasilkan distribusi **P(X | Y = 1)** dengan hasil: X=0 sebesar 0.667, X=1 sebesar 0.167, dan X=2 sebesar 0.167, yang berarti nilai **X = 0** memiliki probabilitas tertinggi ketika diketahui bahwa **Y = 1**.

```
In [21]: query_variable = 'Cavity'
         evidence = dict(Toothache=True)
         ans = enumerate_joint_ask(query_variable, evidence, full_joint)
         (ans[True], ans[False])
```

```
Out[21]: (0.6, 0.39999999999999997)
```

Potongan kode ini digunakan untuk menghitung distribusi probabilitas bersyarat dari variabel **Cavity** dengan kondisi bahwa **Toothache = True**, menggunakan fungsi **enumerate_joint_ask** yang telah dijelaskan sebelumnya. Variabel **query_variable** ditetapkan sebagai **'Cavity'**, sedangkan bukti atau *evidence* diberikan dalam bentuk dictionary **dict(Toothache=True)**. Pemanggilan fungsi **enumerate_joint_ask(query_variable, evidence, full_joint)** akan menghasilkan sebuah objek distribusi probabilitas yang berisi nilai **P(Cavity=True | Toothache=True)** dan **P(Cavity=False | Toothache=True)**. Hasilnya disimpan dalam variabel **ans**, dan baris **(ans[True], ans[False])** digunakan untuk menampilkan kedua nilai tersebut secara berurutan. Berdasarkan data dari *full_joint* distribution, nilai yang diperoleh biasanya mendekati (0.6, 0.4), yang berarti bahwa ketika seseorang mengalami sakit gigi, kemungkinan ia memiliki kavitas gigi sekitar 60%, sedangkan kemungkinan tidak memilikinya sekitar 40%.

Kesimpulan

Melalui serangkaian percobaan dan implementasi pada praktikum ini, konsep inferensi probabilistik menggunakan **Full Joint Distribution** telah berhasil diterapkan. Proses diawali dengan pembuatan kelas **ProbDist** dan **JointProbDist** untuk merepresentasikan distribusi probabilitas tunggal maupun gabungan. Selanjutnya, fungsi **enumerate_joint** digunakan untuk menghitung total probabilitas yang konsisten dengan bukti (*evidence*), sedangkan fungsi **enumerate_joint_ask** dikembangkan untuk menghasilkan distribusi probabilitas bersyarat (*conditional probability distribution*) terhadap suatu variabel kueri dengan melakukan normalisasi.

Hasil percobaan menunjukkan bahwa pendekatan ini dapat digunakan untuk melakukan inferensi, seperti menghitung **P(Cavity=True | Toothache=True)**, yang nilainya mendekati 0.6. Artinya, jika seseorang mengalami sakit gigi, maka peluang ia memiliki kavitas gigi sekitar 60%. Nilai ini diperoleh dengan menjumlahkan seluruh kombinasi nilai

variabel tersembunyi yang relevan, sesuai dengan prinsip enumerasi pada distribusi probabilitas gabungan penuh.

Secara keseluruhan, praktikum ini memperlihatkan bagaimana teori probabilitas dan ketidakpastian dapat diimplementasikan secara komputasional untuk melakukan **reasoning berbasis bukti**, yang menjadi dasar penting dalam bidang **Kecerdasan Komputasional** dan **Artificial Intelligence** pada umumnya.